

Bags, Vectors & Transformers

Two Bonus Topics: fastText & Embedding Features · Day 2, Bonus

Denise J. Roth, PhD Candidate

Strategic Communication Group · Wageningen University & Research

One Problem Static Embeddings Still Have

Word2Vec and GloVe learn **one vector per word in their vocabulary**. But what happens when they meet a word they **never saw during training**?

“policy”, “economy”
in vocabulary ✓

“deglobalisation”, a
typo, a new hashtag
not in vocabulary ✗

A standard static model has **no vector at all** for an unknown word. It is simply dropped — exactly the **out-of-vocabulary** problem we first met with bag-of-words. New words, rare words, typos, and morphological variants all fall through.

fastText: Words Are Made of Pieces

fastText (Bojanowski et al., 2017) fixes this with a simple idea: don't embed whole words only — embed their **subword pieces** (character n-grams) too.

“where”



<wh whe her ere re>

A word's vector is built up from the vectors of its **character n-grams**. So even a word the model never saw as a whole can still get a representation — assembled from pieces it *has* seen.

The payoff: “deglocalisation” shares pieces with “globalisation”, “global”, “-isation”. fastText can build a sensible vector for it even if it never appeared in training. **No more empty hands for unknown words.**

When Does This Matter?

fastText shines for

- **Morphologically rich** languages (Dutch, German, Finnish)
- **Noisy text**: social media, typos, hash-tags
- **Rare / new** words
- Smaller corpora

Good to know

- Otherwise works just like Word2Vec
- Same skip-gram / CBOW training
- Pre-trained fastText vectors exist for **157 languages**
- Still **static** — one vector per word in context-free use

For a Dutch-language or social-media corpus, fastText is often a better default than plain Word2Vec or GloVe.

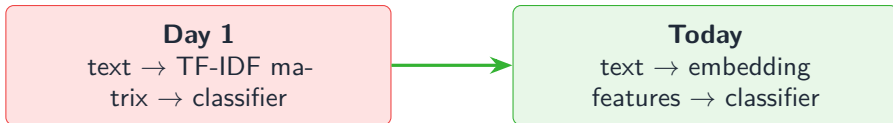
Takeaway

- ✓ Plain static embeddings **drop words they never saw** (the OOV problem)
- ✓ **fastText** represents words as **subword pieces**, so it can build vectors for unseen words
- ✓ Especially useful for **morphologically rich languages** and **noisy text**
- ✓ It is still a **static** embedding — it does not solve the *context* problem (“bank” still gets one vector)

In short: fastText patches the **vocabulary** gap in static embeddings. The **context** gap still waits for contextual models.

Bringing It Full Circle

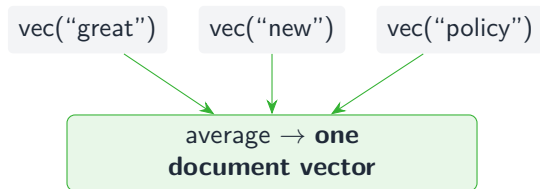
On Day 1 we built **supervised classifiers** on top of a **bag-of-words / TF-IDF** matrix. Now we have a richer representation — **embeddings**. Can we swap one in for the other?



The classifier stays the same (logistic regression, say). All that changes is **how we turn each document into numbers**. This is the payoff of everything we have learned about embeddings.

From Word Vectors to a Document Vector

A classifier needs **one vector per document**. Embeddings give us one vector per **word**. The simplest bridge (from earlier): **average** the word vectors.



Each document becomes a single dense vector (say 300 numbers) — ready to feed into exactly the same classifiers as Day 1. Optionally weight the average by **TF-IDF** so distinctive words count more.

Embeddings vs. Bag-of-Words as Features

Why embeddings can help

- **Dense** & low-dimensional (300 vs. tens of thousands)
- Captures **meaning**: “car” and “auto-mobile” look similar
- Can generalize from **fewer examples**
- Handles words unseen in *your* training set (if in the embeddings)

But watch out

- Averaging **loses word order** (again!)
- Not always better than TF-IDF — **test it**
- Less **interpretable** (no clear “top words”)
- Inherits any **bias** in the embeddings

On short, clean text TF-IDF is a very strong baseline. Embeddings tend to win when meaning and generalization matter most.

Takeaway

- ✓ Embeddings can replace **TF-IDF as the features** feeding a classifier
- ✓ Turn each document into one vector by **averaging** its word vectors (optionally TF-IDF weighted)
- ✓ Upsides: dense, meaning-aware, generalizes from less data
- ✓ Downsides: loses order, less interpretable, and **not automatically better** — always compare against the TF-IDF baseline

The honest rule: embeddings are a powerful *option*, not a guaranteed upgrade. In practice you try both and let the **held-out data** decide — the same discipline from Day 1.